

MOO

Prof. Walter Seiffert

An abstract graphic featuring a horizontal blue streak with a motion blur effect, overlaid with concentric circles and radial lines in yellow and orange, resembling a target or a stylized eye.

Modelagem
orientada
a objetos
com a **UML**

Apresentação do professor



Pós-doutorando em Computação (Universität Munchen)

Doutor em Computação (UFAM/ Universität Stuttgart)

Mestre em Computação (UFAM)

Pós-graduado em Software Livre (UEA)

Pós-graduado em Gerencia de Projetos (UGF)

Graduado em Engenharia da Computação (UTAM)

Graduado em Matemática (UNICAMP)

Graduado em Música (UFPA)

Canais de Comunicação

- Código da turma: 5d2rrh2
- Matéria: MOO



Ementa

Princípios de modelagem software. Modelagem orientada a objetos. A linguagem UML. Arquiteturas de software. Padrões de Projeto. Verificação e Validação de modelos UML.

Introdução

- Software é um componente essencial de qualquer organização
- ❑ e em nossas vidas...
- Desenvolver software é uma tarefa complexa
- São parte de sistemas computacionais igualmente complexos
- ❑ Interface, hardware, procedimentos organizacionais, pessoas, etc.

Introdução

- Crise do software (anos 70)
 - Demanda superior à capacidade de desenvolvimento
 - Custos e prazos raramente cumpridos
 - Baixa “qualidade” dos produtos
- “Engenharia” de Software surge para tentar superar esta crise
 - Estudo de técnicas, metodologias e ferramentas com o objetivo de proporcionar a construção de sistemas com qualidade

Desenvolvimento de Sistemas OO

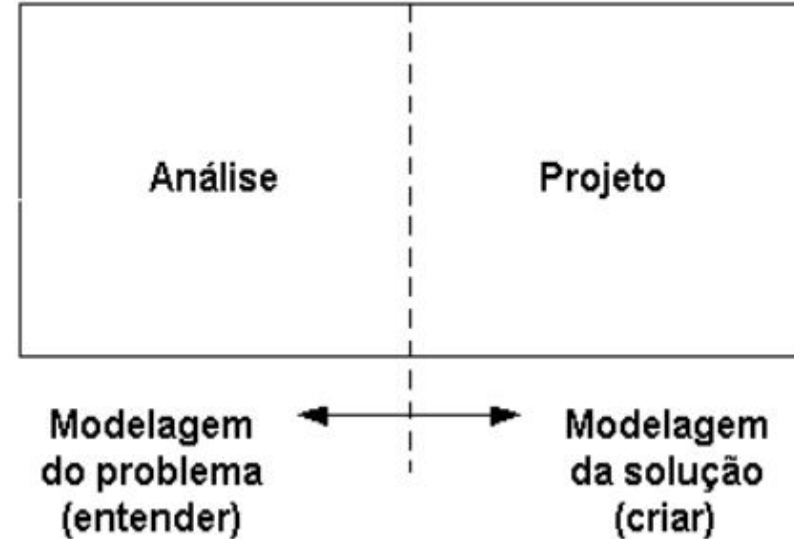
- Aprender as técnicas de orientação a objetos e uma linguagem é apenas parte do processo
 - Para desenvolver bem em OO é preciso saber pensar e organizar um sistema
 - Por que será feito?
 - O que deve ser feito?
 - Como deve ser feito?
 - Como eu posso organizar melhor o meu projeto?
 - Quais as responsabilidades das classes e objetos que eu vou construir?
 - Qual a melhor forma de implementar um determinado método?

Desenvolvimento de Sistemas OO

- Normalmente, a “engenharia” do sistema é feita seguindo um Processo de Desenvolvimento
- Um processo engloba um conjunto de atividades visando compreender, analisar, projetar, desenvolver, testar, implantar, gerenciar e manter um sistema
 - Descreve claramente o que deve ser feito em cada atividade para produzir software de qualidade
- Iremos utilizar uma versão ágil do Processo Unificado (PU), conforme definido no livro de Wazlawick
- Análise e projeto são atividades importantes e essenciais em qualquer processo
 - Mas, afinal, o que é Análise e o que é Projeto?

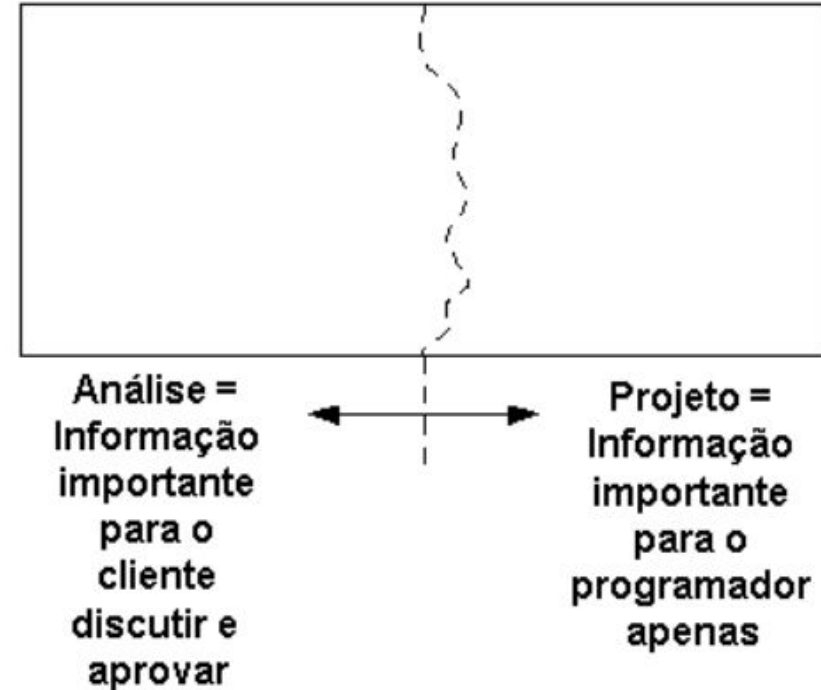
Primeira definição

- **Análise:**
 - modela o problema
 - Atividades necessárias para entender o domínio do problema (o que deve ser feito)
 - Enfatiza a investigação do problema e dos seus requisitos
- **Projeto:**
 - modela a solução satisfazendo os requisitos
 - Atividades de criação (como pode ser feito)
 - Em última instância, projetos podem ser implementados



Segunda Definição

- A análise consiste em todas as atividades feitas com ou para o conhecimento do cliente
- A informação produzida é aquela que o cliente deve discutir e aprovar
- O projeto inclui as atividades que resultam em informação que interessa apenas ao programador



Desenvolvimento de Sistemas OO

- Não há definição que isole claramente Análise e Projeto
- A segunda forma de definir abre a possibilidade de, ao ser apresentado pelo cliente, um modelo de análise possa incluir alguma (pequena) discussão da solução
 - Exemplo: interação do usuário com a interface do sistema
- Podem ser resumidos na frase:
 - Faça a coisa certa (análise) e faça certo a coisa (projeto)

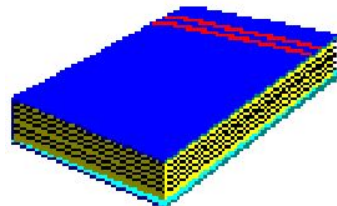
Desenvolvimento de Sistemas OO

- Durante a Análise Orientada a Objetos o objetivo é encontrar e descrever os objetos (ou conceitos) no domínio do problema
 - Ex.: em um sistema de biblioteca, conceitos incluem Livro, Biblioteca, Usuário, Empréstimo, etc.
- O **Projeto Orientado a Objetos** enfatiza a definição dos objetos de software e como eles colaboram para a satisfação dos requisitos
 - Ex.: Na biblioteca, o objeto Livro pode ter um atributo título e o método obterCapítulo. Um Usuário pode ter vários livros emprestados, etc.

Desenvolvimento de Sistemas OO

conceito de domínio

representação dos
conceitos na análise



representação numa
linguagem de
programação orientada a
objeto

```
public class Livro  
{  
    public void print();  
  
    private String título;  
}
```

Desenvolvimento de Sistemas OO

- Todos os sistemas podem ser descritos sob diferentes aspectos, com a utilização de modelos distintos, onde cada modelo será, portanto, uma abstração específica do sistema.
- Os modelos podem ser estruturais, dando ênfase à organização do sistema, ou podem ser comportamentais, dando ênfase à dinâmica do sistema.

Desenvolvimento de Sistemas OO

- De acordo com Booch, Rumbaugh e Jacobson [1], há quatro objetivos principais para se criar modelos:
 1. Eles ajudam a visualizar o sistema como ele é ou como desejamos que ele seja;
 2. **Eles permitem especificar a estrutura ou o comportamento de um sistema;**
 3. Eles proporcionam um guia para a construção do sistema;
 4. **Eles documentam as decisões tomadas no projeto.**

Desenvolvimento de Sistemas OO

- Através dos modelos, conseguimos obter múltiplas visões do sistema, particionando a complexidade do sistema para facilitar sua compreensão, e atuando como meio de comunicação entre os participantes do projeto.
- Portanto, uma linguagem de modelagem padronizada, tal como a UML, é fundamental para a construção e o entendimento de bons modelos.

Princípios de modelagem

- Segundo Booch, Rumbaugh e Jacobson [1] há quatro princípios de modelagem, os quais são citados a seguir.
- A escolha dos modelos a serem criados tem profunda influência sobre a maneira como um determinado problema é atacado e como uma solução é definida.
- Em relação aos softwares, a escolha de modelos poderá ser modificada, de maneira significativa, de acordo com a visão de mundo do projetista. Projetistas distintos podem criar modelos bastante variados entre si, dados os mesmos requisitos do software. Um ponto importante é que cada visão de mundo conduz a um tipo diferente de sistema, com custos e benefícios diversos. A visão de mundo, no caso do desenvolvimento de software, refere-se à experiência dos desenvolvedores e às tecnologias que eles conhecem.

Princípios de modelagem

- Cada visão de mundo pode gerar modelos de sistema diferentes (as tecnologias e técnicas utilizadas nos sistemas podem ser diferentes). O modelo do sistema deve adequar-se não só aos requisitos do mesmo, como também aos conhecimentos da equipe de desenvolvimento. Criar projetos que utilizam tecnologias desconhecidas pela equipe pode atrapalhar o andamento. Os desenvolvedores teriam que aprender essas novas tecnologias para utilizar seus recursos.
- Há várias técnicas existentes e cada uma delas é adequada para um determinado caso. Uma técnica pode ser mais eficiente que outra em um caso, o que não significa que ela será a melhor em todos os casos. Dessa forma, saber qual técnica utilizar num determinado momento é um fator crucial para o sucesso do desenvolvimento de software de boa qualidade.

Cada modelo poderá ser expresso em diferentes níveis de precisão.

Princípios de modelagem

- Os melhores tipos de modelos são aqueles que permitem a escolha do grau de detalhamento das informações, dependendo de quem esteja fazendo a visualização e por que deseja fazê-la. **O diagrama de casos de uso**, que será discutido, permite escolher o grau de detalhes que será mostrado.
- Com esse diagrama, podemos ilustrar as funcionalidades do software numa visão de alto nível. E a partir de cada caso de uso elaborado, podemos subdividi-lo em outros casos de uso mais específicos, o que aumenta o nível de precisão do diagrama.

Princípios de modelagem

- **Os melhores modelos estão relacionados à realidade.**
- Todos os modelos simplificam a realidade. Por isso, certifique-se de que sua simplificação não ocultará detalhes importantes.
- De acordo com Booch, Rumbaugh e Jacobson [1], o tendão de Aquiles das técnicas de análise estruturada está no fato de não haver um relacionamento entre o modelo de análise e o modelo de projeto do sistema. Dessa forma, ao longo do tempo, aparecerá uma divergência entre o sistema concebido e o sistema projetado.
- Nos sistemas orientados a objetos, é possível estabelecer uma ligação entre várias partes do sistema, o que facilita bastante o desenvolvimento de sistemas complexos e o bom entendimento dos mesmos.

Princípios de modelagem

- Nenhum modelo único é suficiente. Qualquer sistema não trivial será melhor investigado por meio de um pequeno conjunto de modelos quase independentes.
- Nesse contexto, a expressão “quase independentes” significa modelos que possam ser criados e estudados separadamente, mas que continuam inter-relacionados.

Introdução à linguagem UML

Introdução a UML

- A **UML (*Unified Modeling Language*)** foi desenvolvida por Grady Booch, James Rumbaugh e Ivar Jacobson.
- Cada um deles possuía sua própria sistemática de criar modelos e a UML é a junção dos pontos fortes dessas três abordagens, adicionando novos conceitos e visões de linguagem.

Introdução a UML

- Antes da elaboração da *Unified Modeling Language*, existiam diversos padrões para se criar modelos de software, o que dificultava a elaboração de software por equipes que utilizassem padrões diferentes.
- Esta linguagem é uma maneira de padronizar a modelagem orientada a objetos de forma que qualquer sistema possa ser modelado da maneira apropriada, simples de ser atualizado e compreendido.
- UML é uma linguagem muito expressiva, abrangendo todas as visões necessárias ao desenvolvimento e implantação de sistemas de software em geral.

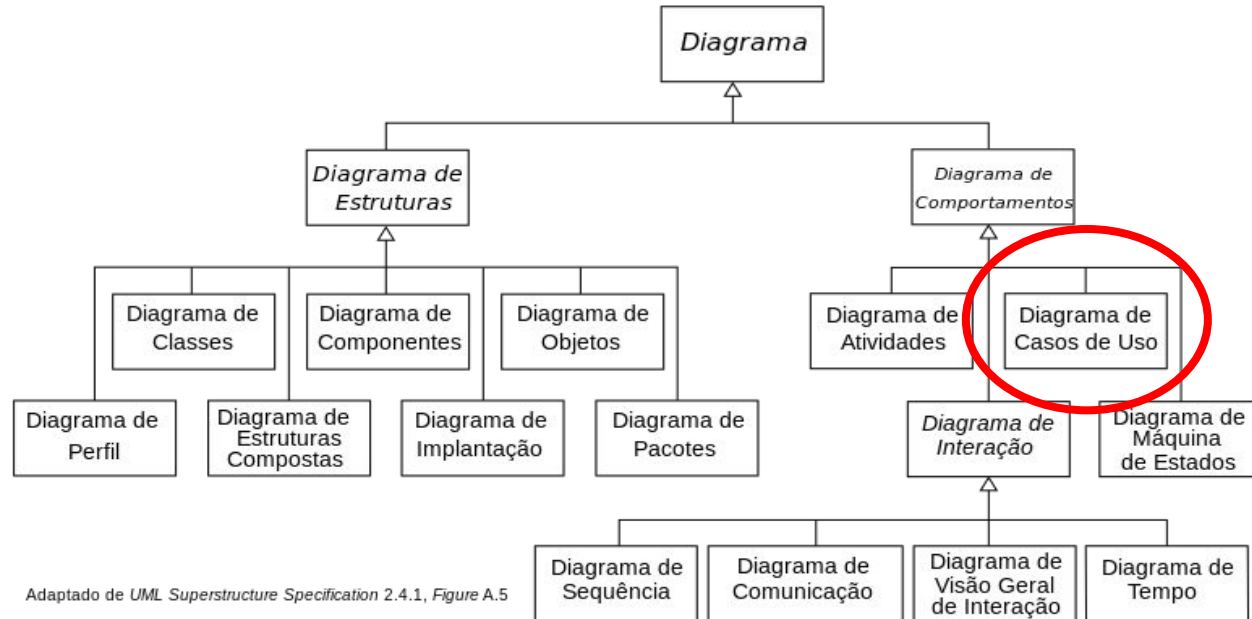
Introdução a UML

- UML é uma linguagem padrão para a elaboração da arquitetura de projetos de software. Ela pode ser empregada para visualização, especificação, construção e documentação de artefatos de software.
- Ela aborda o caráter estático e dinâmico do sistema a ser analisado. Além disso, UML leva em consideração, já no período de modelagem, todas as futuras características do sistema. Tais características estão relacionadas a trocas de mensagens entre as diversas partes do sistema, o padrão arquitetural adotado e os padrões de projeto utilizados no mesmo.

Especificação de requisitos

Diagramas

- São representações gráficas de um conjunto de elementos. São desenhados para visualizar um sistema de diferentes perspectivas.



Adaptado de UML Superstructure Specification 2.4.1, Figure A.5

Diagrama de Casos de Uso

- Os Diagramas de Casos de Uso são utilizados para modelar:
 - O contexto de um sistema, identificando os atores e seus papéis na interação com o sistema;
 - Os requisitos de um sistema, especificando o que o sistema deve fazer (do ponto de vista de seus usuários), sem no entanto se preocupar em como é implementado.
- Os Diagramas de Casos de Uso servem para:
 - Verificar e validar a arquitetura do sistema;
 - Identificar e gerar casos de teste.

Diagrama de Casos de Uso

Considerações:

- Cada diagrama de casos de uso representa graficamente uma visão parcial do sistema.
- O conjunto de diagramas de casos de uso formam a visão de casos de uso completa do sistema.
- Diagramas de Casos de Uso representam uma visão externa ao sistema, servindo de base para a identificação e especificação do conjunto de classes - e suas interações - necessárias para atingir os objetivos e propósitos do sistema.

Diagrama de Casos de Uso

Objetivos:

- Descrever em detalhe a técnica de Modelagem de Requisitos com Casos de Uso
- Apresentar um modelo de documento de requisitos
- Discutir o uso do diagrama de atividades na modelagem de casos de uso

Conceitos Básicos

Conceitos Básicos

- Não-Funcionais
 - Atributos ou qualidades do sistema
- Funcionais
 - Expressam funcionalidades do sistema

Alguns Tipos de Requisitos Não-Funcionais

- Usabilidade (Facilidade de uso)
- Confiabilidade
- Desempenho
- Segurança
- Distribuição
- Adequação a Padrões
- Restrições de Hardware e Software

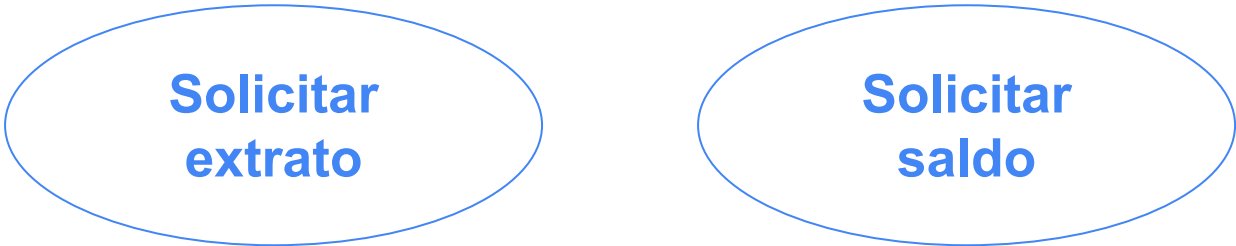
Caso de uso

- É uma forma específica de uso do sistema através da execução de alguma de suas funcionalidades.
- Uma unidade coerente de funcionalidade provida por um sistema, manifestada por uma seqüência de mensagens trocadas entre o sistema e um ou mais usuários externos (representados como atores), junto com as ações executadas pelo sistema.

Caso de uso

- Descrevem o que acontece dentro do sistema.
- Ajudam muito na comunicação entre clientes e desenvolvedores.
- Mostram apenas o que o sistema faz, e não como.
 - Capturam o comportamento pretendido para um sistema, sem a necessidade de especificar como esse comportamento será implementado.

Caso de uso: representação gráfica



**Solicitar
extrato**

**Solicitar
saldo**

UML – Diagrama de Caso de Uso

“Documento narrativo que descreve a sequência de eventos de um ator que usa um sistema para completar um processo.”

- Representa a interação entre um usuário (humano ou sistema) e o sistema.
- Não descreve como o software deverá ser construído, mas sim como ele deverá se comportar quando estiver pronto.
- Corresponde a um conjunto de ações com um objetivo comum.

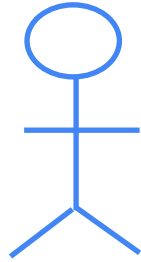
UML – Diagrama de Caso de Uso

- Constituem as entidades que interagem com o ambiente do sistema
 - Pessoas ou outros sistemas (de hardware ou software) que interagem com o sistema em desenvolvimento
- Definem um papel particular
- São sempre externos ao sistema
- O sistema será descrito através de vários casos de uso que são executados por um número de atores

UML – Diagrama de Caso de Uso

- Servem para facilitar o entendimento de um sistema mostrando a sua “visão externa”
- São usados para modelar o contexto de um sistema, subsistema ou classe
- São uma das maneiras mais comuns de documentar os requisitos do sistema
 - Delimitam o Sistema
 - Definem a funcionalidade do sistema

UML – Diagrama de Caso de Uso



Cliente



Caixa

UML – Diagrama de Caso de Uso

- Uma mesma pessoa pode desempenhar diferentes papéis



Carlos como
professor



Professor

Carlos como
estudante



Estudante

UML – Diagrama de Caso de Uso

- A coleção dos use cases deverá especificar todas as formas existentes de uso do sistema



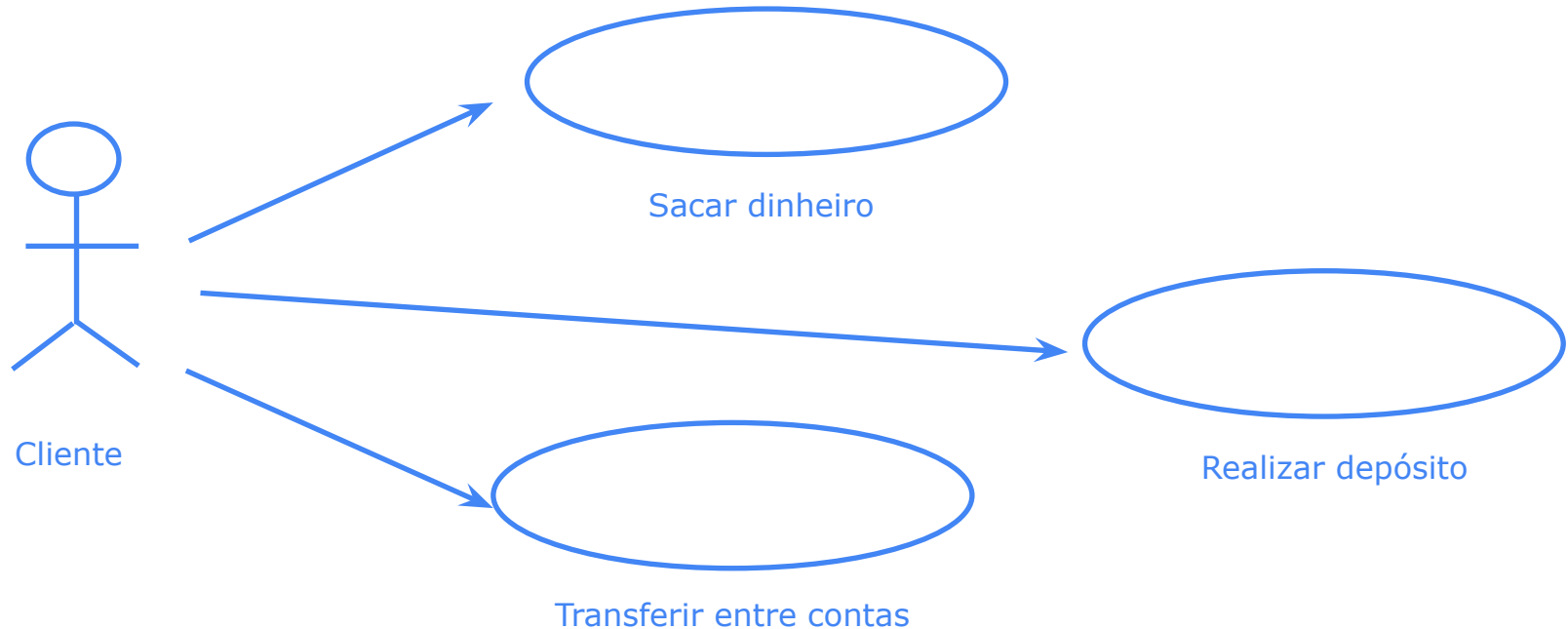
**Matricular
aluno**

**Solicitar
histórico**

**Verificar
pré-requisit
os**

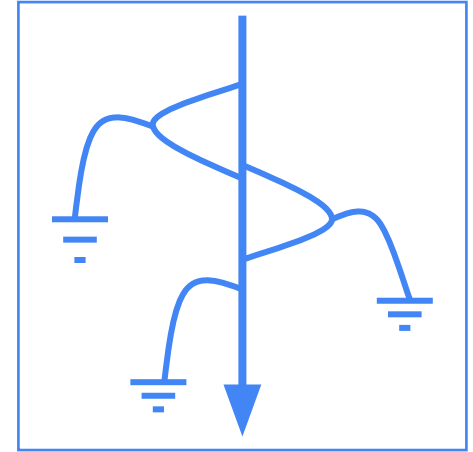
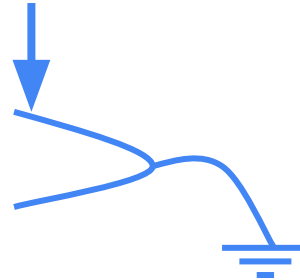
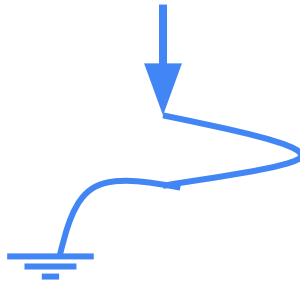
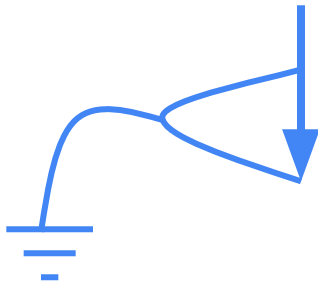
UML – Diagrama de Caso de Uso

Uma associação entre um *ator* e um *caso de uso* indica que há uma comunicação, possivelmente com envio e recepção de mensagens.



Cenários

- Em UML significa um caminho através de um caso de uso.
- Uma instância de um caso de uso
- Exemplo (Sacar dinheiro):
 - Saque com sucesso
 - Tentativa de saque MAS senha incorreta
 - Tentativa de saque MAS saldo insuficiente



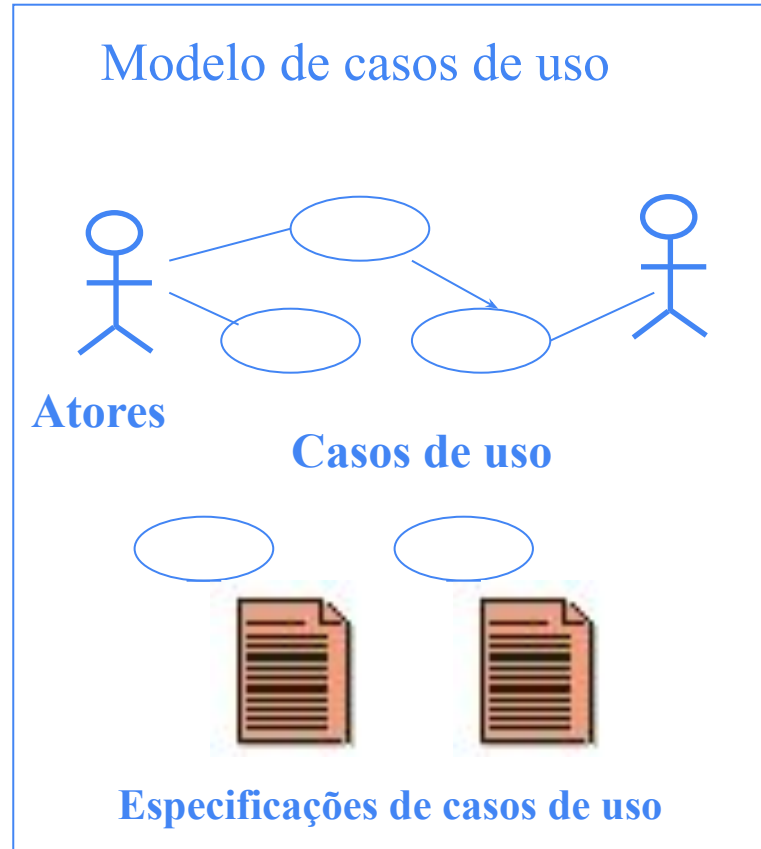
Requisitos Não-Funcionais x Casos de Uso

- Requisitos não funcionais podem ser:
 - Específicos: associados a um caso de uso específico
 - Genéricos: associados a vários casos de uso ou ao sistema com um todo
- Para serem atendidos podem gerar novos casos de uso

Requisitos Não-Funcionais x Casos de Uso

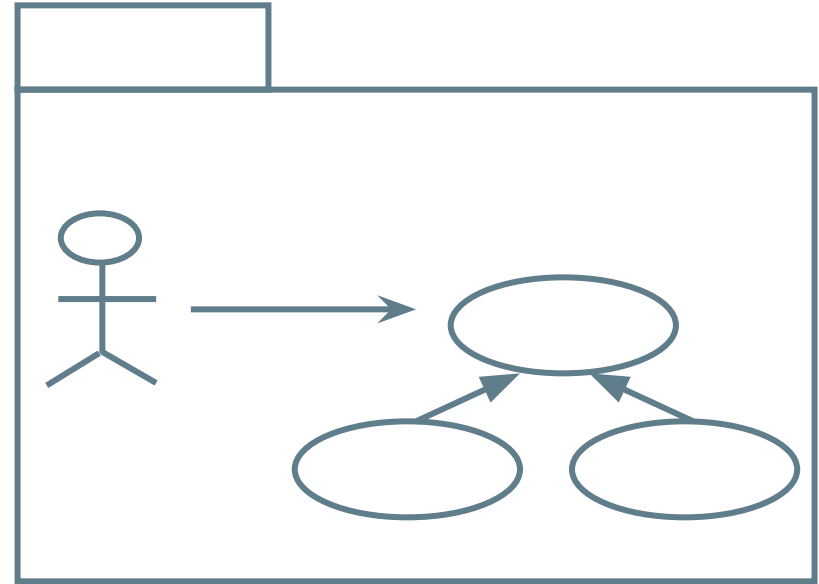
- Identificador do caso de uso
- Breve Descrição
- Ator (opcional)
- Prioridade
- Pré condições
- Pós condições
- Fluxo de eventos:
 - Fluxo de eventos principal
 - Fluxos secundários: alternativos e de exceção
- Requisitos Não-Funcionais Específicos

Requisitos Não-Funcionais x Casos de Uso



Pacotes de Casos de Uso

- Servem para agrupar casos de uso relacionados
- Critérios para agrupamento:
 - ator
 - funcionalidades correlatas
 - processos
 - “um por todos e todos por um”



Um Modelo de Documento de Requisitos

- Introdução
 - Objetivos do documento
 - Público ao qual se destina
 - Termos e acrônimos
 - Referências
- Descrição geral do sistema
 - Abrangência e sistemas relacionados
 - Descrição dos usuários
- Casos de uso
 - Diagrama
 - Especificações
- Requisitos não funcionais genéricos

Ator

- Humano ou entidade.
- Interage com o sistema.
- Iniciam o sistema.
- Fornecem dados.
- Usam as informações do sistema.



Ator

Como encontrar atores e casos de uso?



Como encontrar atores?

- Quem usa o sistema?
- Quem instala/mantém o sistema?
- Quem inicia/desliga o sistema?
- Que outros sistemas usam o sistema?
- Quem recebe informação do sistema?
- Quem provê informação ao sistema?



Ator

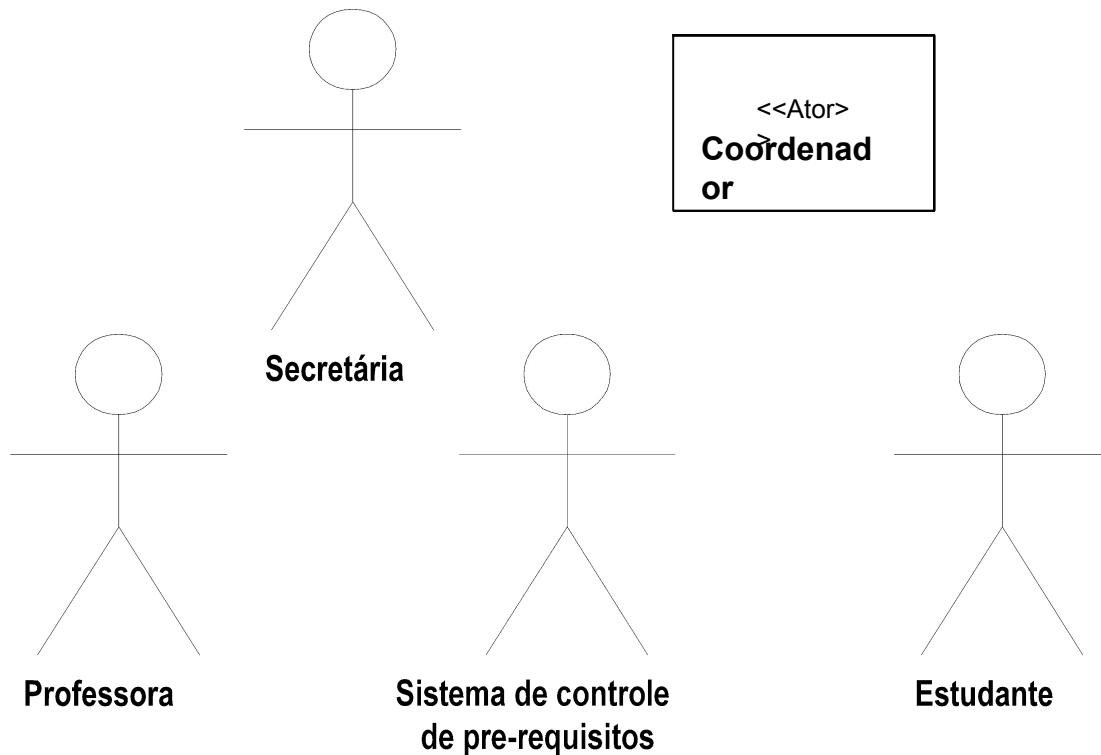
Como encontrar casos de uso?

- Atores são fundamentais para a descoberta dos casos de uso
- Pergunte
 - Que funções o ator vai querer do sistema?
 - O sistema armazena informações? Que informações atores irão criar, ler, atualizar ou apagar?
 - O sistema precisa notificar o ator sobre mudanças no seu estado interno?
 - Existe algum evento externo que o sistema precisa saber? Que ator informa o sistema desses eventos?

Como encontrar casos de uso?

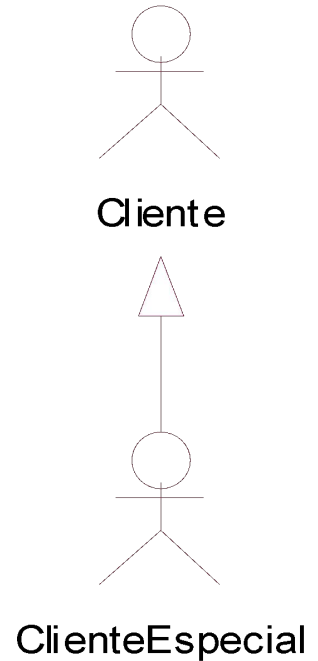
- O sistema será descrito através de vários *use cases* que são executados por um número de *atores*
- Atores constituem as entidades do ambiente do sistema
- São pessoas ou outros subsistemas que interagem com o sistema em desenvolvimento

Atores - Notação



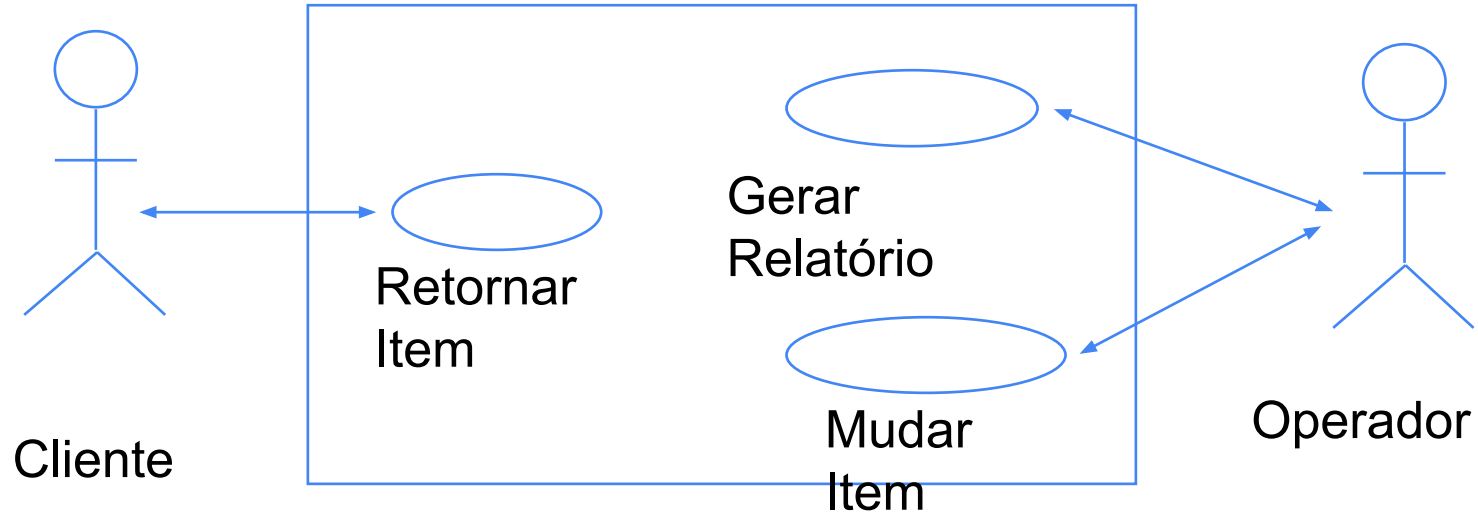
Atores: Especialização

- É possível definir tipos gerais de atores e especializá-los usando o relacionamento de especialização



Especificação Detalhada dos Casos de Uso

Diagrama de Use Case



Expressão de variantes de use case

- Nem sempre é óbvio decidir se uma funcionalidade corresponde a um novo use case. As vezes trata-se de uma variação de um mesmo use case
 - Se as diferenças forem pequenas elas podem ser descritas através de variantes de um mesmo use case
 - Se as diferenças são grandes elas devem ser descritas como use cases separados

Expressão de variantes de use case

- Use Case Retornar item

Fluxo principal de eventos:

- Quando o *cliente* depositar os seus itens, ele/ela irá pressionar o *botão recibo* para obter o recibo. O recibo impresso irá listar os itens depositados, seus totais e o valor a ser pago ao *cliente*

Expressão de variantes de use case

- Use Case Retornar item

Fluxo excepcional de eventos:

- Quando o cliente retorna um item ele é medido pelo sistema. A medição é usada para determinar que tipo de lata, garrafa ou gradeado foi depositado. Se aceito o total do cliente será incrementado. Se não for aceito, apresentar mensagem 'NÃO É VALIDA'

Quando e por que realizá-las?

- Quando?
 - após fazer levantamento dos principais casos de uso do sistema
- Por que?
 - descrever detalhes dos casos de uso
 - descrever fluxos de eventos e outras propriedades
 - uniformizar entendimento entre clientes, usuários e equipe de desenvolvimento

Identificação do caso de uso

- Deve ser única!
- Não deve mudar nunca
- Pois casos de uso podem ser referenciados por seu identificador

Breve descrição do caso de uso

- Dar uma idéia do propósito do caso de uso, do seu objetivo
- Deve ser feita ao se identificar o caso de uso, para evitar mal-entendidos
- 2 ou 3 linhas!

Prioridade de casos de uso

- Essencial para gerenciar os requisitos
- É preciso definir prioridade de todos os casos de uso
- Exemplos de prioridade:
 - Essencial
 - Importante
 - Desejável

Pré e pós condições

- O que deve ser verdade antes e depois da realização do caso de uso!

Pré e pós condições: exemplos

- Caso de uso “Entregar pedido”
 - Pré condição: os itens do pedido devem existir em estoque
 - Pós condição: os itens enviados devem ser abatidos do estoque
- Caso de uso “Recadastrar CPF”
 - Pré condição: o usuário deve possuir um CPF
 - Pós condição: a situação do contribuinte é atualizada

Fluxo de eventos básico/principal

- Série de passos que compõem um caso de uso
- Concentre-se inicialmente na funcionalidade básica/central do caso de uso
- Pense nos fluxos secundários depois!

Exemplo de um fluxo básico

- Caso de uso “Sacar dinheiro”
 1. O cliente passa o seu cartão
 2. Digita sua senha
 3. Digita o valor do saque
 4. O sistema verifica se há saldo suficiente
 5. O saldo é debitado da conta do cliente
 6. O dinheiro é entregue ao cliente

Exemplo de um fluxo básico

- Caso de uso “Sacar dinheiro”
- MAS...
 - E se a senha não conferir?
 - E se não houver saldo?
 - E se não houver dinheiro suficiente na máquina?

Fluxos secundários

- Só devem ser analisados e descritos após a descrição dos fluxos básicos.
 - situações especiais (ex: desconto para um cliente)
 - situações de erro

Atividade

Atividade

A pouco tempo atrás o Covid-19 restringiu o deslocamento das pessoas para suas atividades na rua, entre eles o acesso a ambientes hospitalares.

Uma pessoa ao acessar o setor de Urgencia/ Emergencia faz um procedimento chamado TRIAGEM. Ali, ela informa nome, idade, sexo, ID e em seguida informa o que está sentindo.

Uma forma de reduzir a sua permanência no ambiente hospitalar é fornecendo um chatBot

Atividade

As perguntas a serem feitas ao paciente são:

Vc viajou recentemente ou teve contato com alguém que viajou?

Vc está sentindo febre? Mediu?

Vc tem tosse?

Vc está sentindo alguma dor corporal? Onde?

A quanto tempo vc está assim?

Se a resposta de alguma das perguntas for SIM, combinada com a primeira pergunta, o chatBot deve indicar que o paciente evite o contato com outras pessoas, use mascara, luvas. Deve ser pedido o endereço para a sua busca por técnicos.

Especificando Casos de Uso

Especificando Casos de Uso

- **Seqüência de Eventos**

Seqüência Típica de Eventos

Ação do Ator

Ações numeradas de ator

Resposta do Sistema

Descrição numerada
das respostas do
sistema

Seqüências Alternativas

Alternativas que podem surgir
por número de linha:
descrição de exceções.

Especificando Casos de Uso

- **Identificação:** UC1
- **Caso de uso:** Sacar dinheiro no caixa eletrônico
- **Ator:** Cliente
- **Pré-Condições:** o Cliente possui cartão do banco e senha cadastrada.
- **Pós-Condições:** lançada a transação na conta do Cliente, atualizado o saldo da conta corrente e liberado o dinheiro.
- **Sequência Típica de Eventos (Fluxo Básico):**

Ação do ator

1. Este caso de uso começa quando o Cliente realiza a leitura do cartão do banco no caixa eletrônico
2. O Cliente informa a sua senha
3. O Cliente informa o valor do saque

Resposta do sistema

3. O sistema valida a conta corrente e senha do Cliente, autorizando a operação
5. O sistema autoriza o saque e lança o débito na conta corrente do Cliente
6. O sistema libera o dinheiro

Especificando Casos de Uso

Forma Alternativa de Apresentação

Identificação: UC1

Caso de uso: Sacar dinheiro no caixa eletrônico

Ator: Cliente

Pré-Condições: o Cliente possui cartão do banco e senha cadastrada.

Pós-Condições: lançada a transação na conta do Cliente, atualizado o saldo da conta corrente e liberado o dinheiro.

Seqüência Típica de Eventos (Fluxo Básico):

1. Este caso de uso começa quando o Cliente realiza a leitura do cartão do banco no caixa eletrônico
2. O Cliente informa a sua senha
3. O sistema valida a conta corrente e senha do Cliente, autorizando a operação.
4. O Cliente informa o valor do saque
5. O sistema autoriza o saque e lança o débito na conta corrente do Cliente
6. O sistema libera o dinheiro

Especificando Casos de Uso

Seqüências Alternativas (Fluxos Alternativos):

3a. Cliente Inválido:

1. O sistema não reconhece a conta corrente e senha do Cliente como válida
2. A operação é cancelada

5a: Fundos Insuficientes:

1. O sistema não autoriza o valor solicitado para saque pelo Cliente
2. A operação é cancelada

Requisitos Não-Funcionais

- Resposta do sistema deve ocorrer em no máximo 30 seg em 90 % dos casos

Especificando Casos de Uso

Casos de Uso: deve ser definido um caso de uso para cada objetivo de usuário de um ator.

- O nome do caso de uso é similar ao objetivo de usuário.
 - Exemplo:
 - Ator: Cliente
 - Objetivo: Retirar dinheiro de sua conta corrente
 - Caso de Uso: Sacar dinheiro no caixa eletrônico
- Exceção: casos de uso para tratamento de informações persistentes do sistema – CRUD (create, retrieve, update, delete).
 - Estes casos de uso podem ser comumente identificados por Atualizar <X>, como por exemplo o caso de uso Atualizar Conta Corrente.

Priorizando Casos de Uso

- Esta atividade objetiva estabelecer uma priorização entre os casos de uso de forma a estabelecer quais deles deverão ser desenvolvidos nas primeiras iterações e quais podem ser desenvolvidos em iterações posteriores.
- Categorias:
 - Casos de uso primários: representam processos comuns principais.
 - Casos de uso secundários: representam processos menos importantes ou raros.
 - Casos de uso opcionais: representam processos que podem ser desconsiderados.

Especificando Casos de Uso

Ramificações

- Estrutura de Notação:
 - Caracteriza situações em que existem duas ou mais opções de continuidade no fluxo de uma determinada seção.
 - Dentro da Seqüência Típica de Eventos de uma seção indique desvios para subseções;
 - Escreva uma subseção para cada desvio usando novamente uma Seqüência Típica de Eventos.

Atividade 2

Atividade

Crie o controle de processos de caso de uso para a atividade 1

Referências

- WAZLAWICK, Raul S. Análise e Projeto de Sistemas de Informação Orientados a Objetos. 2ª ed. Rio de Janeiro : Elsevier, 2011.
- LARMAN, Craig. Utilizando UML e Padrões. 3ª ed. Porto Alegre : Bookman, 2007.
- CAMPOS, Edmilson. Introdução a Análise e Projeto Orientados a Objetos. Disponível em:
<http://edmilsoncampos.files.wordpress.com/2011/07/aula-03-introduc3a7c3a3o-c3a0-anc3a1lise-e-projeto-orientado-c3a0-objetos.pdf>
- SAUVÉ, Jacques. Notas de aula: Introdução a Análise e Projeto Orientados a Objetos. Disponível em: <http://www.dsc.ufcg.edu.br/~jacques/cursos/map/html/intro/intro.htm>